

Regulatory Document RAG Pipeline - AWS Setup Reference

Debashish

September 2026

AWS Setup & Glue Pipeline - Reference Guide

This document covers the full setup path for the Regulatory Document Q&A RAG project: from configuring AWS CLI access, through creating IAM roles, setting up environment/git config, to running the Federal Register ingestion and Glue ETL pipeline end-to-end.

1. Create an IAM User (Programmatic Access)

Never use your AWS root account for day-to-day work or scripts. Create a dedicated IAM user instead.

Steps:

1. AWS Console → **IAM** → **Users** → **Create user**
2. Name: reg-doc-rag-dev
3. Do **not** enable console access - this user only needs programmatic (CLI/SDK) access
4. Attach permissions (managed policies, scoped to what the project needs):
 - AmazonS3FullAccess
 - AWSGlueConsoleFullAccess
 - AmazonDynamoDBFullAccess
 - AmazonRedshiftFullAccess (add later, once you reach the analytics layer)
5. Click **Create user**

Generate the access key:

1. Open the user → **Security credentials** tab
 2. Scroll to **Access keys** → **Create access key**
 3. Select use case: **Command Line Interface (CLI)**
 4. Copy both values immediately - the secret key is shown only once:
 - Access Key ID (starts with AKIA...)
 - Secret Access Key
-

2. Configure AWS CLI Locally

```
aws configure
```

You'll be prompted for:

AWS Access Key ID: [paste your access key]
AWS Secret Access Key: [paste your secret key]

Default region name: us-east-1
Default output format: json

Verify the CLI is configured correctly:

```
aws configure list
```

Shows your current profile settings (keys are masked) without exposing the secret.

Verify the credentials actually work (real API call):

```
aws sts get-caller-identity
```

Expected output:

```
{
  "UserId": "AIDAR3HU0WHBSFFUXY6VG",
  "Account": "127214203331",
  "Arn": "arn:aws:iam::127214203331:user/reg-doc-rag-dev"
}
```

If this returns successfully, your CLI is authenticated and any local boto3 code using `boto3.client("s3")` (no explicit keys) will automatically use these credentials.

List all configured profiles (if you have more than one):

```
aws configure list-profiles
```

3. Create the IAM Role for AWS Glue

This is a **separate identity** from your IAM user. Your user runs CLI commands and deploys jobs; the Glue **role** is what the Glue *service* assumes internally while your job is actually executing.

Steps:

1. IAM → **Roles** → **Create role**
2. Trusted entity type: **AWS service**
3. Use case: **Glue** - this auto-generates the correct trust policy allowing `glue.amazonaws.com` to assume the role
4. Attach permissions:
 - AWSGlueServiceRole (AWS managed baseline policy)
 - AmazonS3FullAccess
 - AmazonDynamoDBFullAccess

(For production, scope these down to your specific bucket/table ARNs instead of full access - good interview talking point on least-privilege IAM.)

5. Name it: `AWSGlueServiceRole-reg-doc-rag`
6. Click **Create role**

Copy the Role ARN from the role's Summary page:

```
arn:aws:iam::127214203331:role/AWSGlueServiceRole-reg-doc-rag
```

You'll need this exact string in your `.env` file (see Section 5).

4. Create AWS Resources (S3 Bucket + DynamoDB Table)

Create the S3 bucket (must be globally unique):

```
aws s3 mb s3://reg-doc-rag --region us-east-1
```

Create the DynamoDB metadata table:

```
aws dynamodb create-table \  
  --table-name reg-doc-metadata \  
  --attribute-definitions AttributeName=document_id,AttributeType=S \  
  \  
  --key-schema AttributeName=document_id,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST
```

5. Configure .env (Secrets - Never Committed to Git)

Create the file in your project root:

```
touch .env
```

Contents:

```
AWS_DEFAULT_REGION=us-east-1  
S3_RAW_BUCKET=reg-doc-rag  
S3_SCRIPTS_BUCKET=reg-doc-rag  
GLUE_ROLE_ARN=arn:aws:iam::127214203331:role/AWSGlueServiceRole-reg-  
doc-rag  
DYNAMODB_TABLE=reg-doc-metadata  
FEDERAL_REGISTER_API_KEY=
```

Notes:

- S3_RAW_BUCKET and S3_SCRIPTS_BUCKET can point to the same bucket - they're separate variable names for separate *purposes* (raw ingested data vs. uploaded Glue script), not separate buckets. Splitting them later is an option for tighter per-purpose IAM permissions, not a requirement.
- FEDERAL_REGISTER_API_KEY can stay blank - the endpoints used in this project don't require one.
- If you use plain `aws` configure credentials (recommended), you do **not** need `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` in `.env` at all - `boto3` picks up `~/.aws/credentials` automatically.

Commit a template instead of the real file, so the repo documents required variables without leaking values:

```
# .env.example (safe to commit)  
AWS_DEFAULT_REGION=us-east-1  
S3_RAW_BUCKET=  
S3_SCRIPTS_BUCKET=  
GLUE_ROLE_ARN=  
DYNAMODB_TABLE=  
FEDERAL_REGISTER_API_KEY=
```

6. Configure .gitignore

Create the file in your project root:

```
touch .gitignore
```

Contents:

```
# Python
__pycache__/
*.pyc
.venv/
venv/
*.egg-info/

# AWS / secrets
.env
.aws/
*.pem
credentials.json
terraform.tfstate*

# Data / large files
data/raw/
*.parquet
*.csv
.ipynb_checkpoints/

# IDE
.vscode/settings.json
.DS_Store
```

Verify .env is actually ignored:

```
git check-ignore -v .env
```

Expected output (confirms it's protected):

```
.gitignore:9:.env      .env
```

Important: .gitignore only prevents *new* untracked files from being added. If a secret file was already committed before being added here, it must be removed from git history separately (git rm --cached) and the exposed key **rotated** in AWS.

7. Install Python Dependencies

```
pip install boto3 requests python-dotenv
```

(PySpark/Glue libraries are **not** needed locally - the Glue ETL script only runs inside AWS's managed Glue environment, never on your laptop.)

8. Project Folder Structure

```
Data Engineering with AI/
├── ingestion/
│   └── federal_register_ingest.py
├── glue_jobs/
│   ├── federal_register_etl.py
│   └── deploy_glue_job.py
├── .env
├── .env.example
├── .gitignore
└── requirements.txt
```

9. Run the Ingestion Script (Local)

Pulls documents from the Federal Register API and lands raw JSON in S3, partitioned by publication date.

```
cd ingestion
python federal_register_ingest.py --start 2026-08-01 --end 2026-08-31
```

Verify it landed in S3:

```
aws s3 ls s3://reg-doc-rag/raw/federal_register/ --recursive
```

10. Deploy the Glue ETL Job

This uploads `federal_register_etl.py` to S3 and registers/updates the Glue job definition via boto3.

```
cd ../glue_jobs
python deploy_glue_job.py
```

You'll be prompted:

Run the job now? (y/n):

Type y to trigger the run immediately.

11. Monitor the Glue Job Run

Check status via CLI:

```
aws glue get-job-runs --job-name federal-register-etl --max-results 1
```

Look for "JobRunState" - progresses from RUNNING → SUCCEEDED or FAILED.

View logs if it fails:

```
aws logs tail /aws-glue/jobs/output --follow
```

Or monitor visually: AWS Console → **Glue** → **Jobs** → `federal-register-etl` → **Runs** tab.

12. Verify Output

Curated Parquet output in S3:

```
aws s3 ls s3://reg-doc-rag/curated/federal_register/ --recursive
```

Metadata written to DynamoDB:

```
aws dynamodb scan --table-name reg-doc-metadata --max-items 5
```

13. Known Issues Fixed During Setup

Issue	Cause	Fix
	concat_ws	

AnalysisException: expression 'agency_names' is neither present in the group by...	applied directly on an exploded column instead of an aggregated list	Wrap in collect_list(): concat_ws(", ", collect_list(col("agency_names"
ParamValidationError: Unknown parameter in JobUpdate: "Name"	update_job's JobUpdate body doesn't accept a Name key - job name is a separate top-level argument	Removed Name from the shared job_config dict; passed Name=JOB_NAME only to create_job, and JobName=JOB_NAME separately update_job

14. Budget & Cost Safety Net

Set this up once, early, to avoid billing surprises while iterating on Glue jobs.

1. AWS Console → **Billing and Cost Management** → **Budgets** → **Create a budget**
2. Type: **Cost budget**, Period: **Monthly**, Amount: e.g. **\$20-30**
3. Add alert thresholds at 50%, 80%, 100% of the budget
4. Enter your email for notifications

Also enable **Billing Alerts** (Billing preferences → “Receive Billing Alerts”) and create a backup CloudWatch alarm on the EstimatedCharges metric.

Cost watch-outs for this project:

- **Glue** - billed per DPU-hour; don't leave crawlers/jobs running unnecessarily
- **OpenSearch Serverless** - has a minimum compute-unit floor even at low usage
- **Bedrock** - billed per token for embeddings/generation
- **Redshift** - hourly cost while running; stop/delete between sessions, or use Redshift Serverless

Quick Command Reference

```
# --- AWS CLI setup & verification ---
aws configure
aws configure list
aws sts get-caller-identity
aws configure list-profiles

# --- Resource creation ---
aws s3 mb s3://reg-doc-rag --region us-east-1
aws dynamodb create-table --table-name reg-doc-metadata \
  --attribute-definitions AttributeName=document_id,AttributeType=S
\
  --key-schema AttributeName=document_id,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

```
# --- Git safety check ---
git check-ignore -v .env

# --- Run ingestion ---
python ingestion/federal_register_ingest.py --start 2026-08-01 --end
2026-08-31

# --- Deploy & run Glue job ---
python glue_jobs/deploy_glue_job.py

# --- Monitor & verify ---
aws glue get-job-runs --job-name federal-register-etl --max-results
1
aws logs tail /aws-glue/jobs/output --follow
aws s3 ls s3://reg-doc-rag/curated/federal_register/ --recursive
aws dynamodb scan --table-name reg-doc-metadata --max-items 5
```